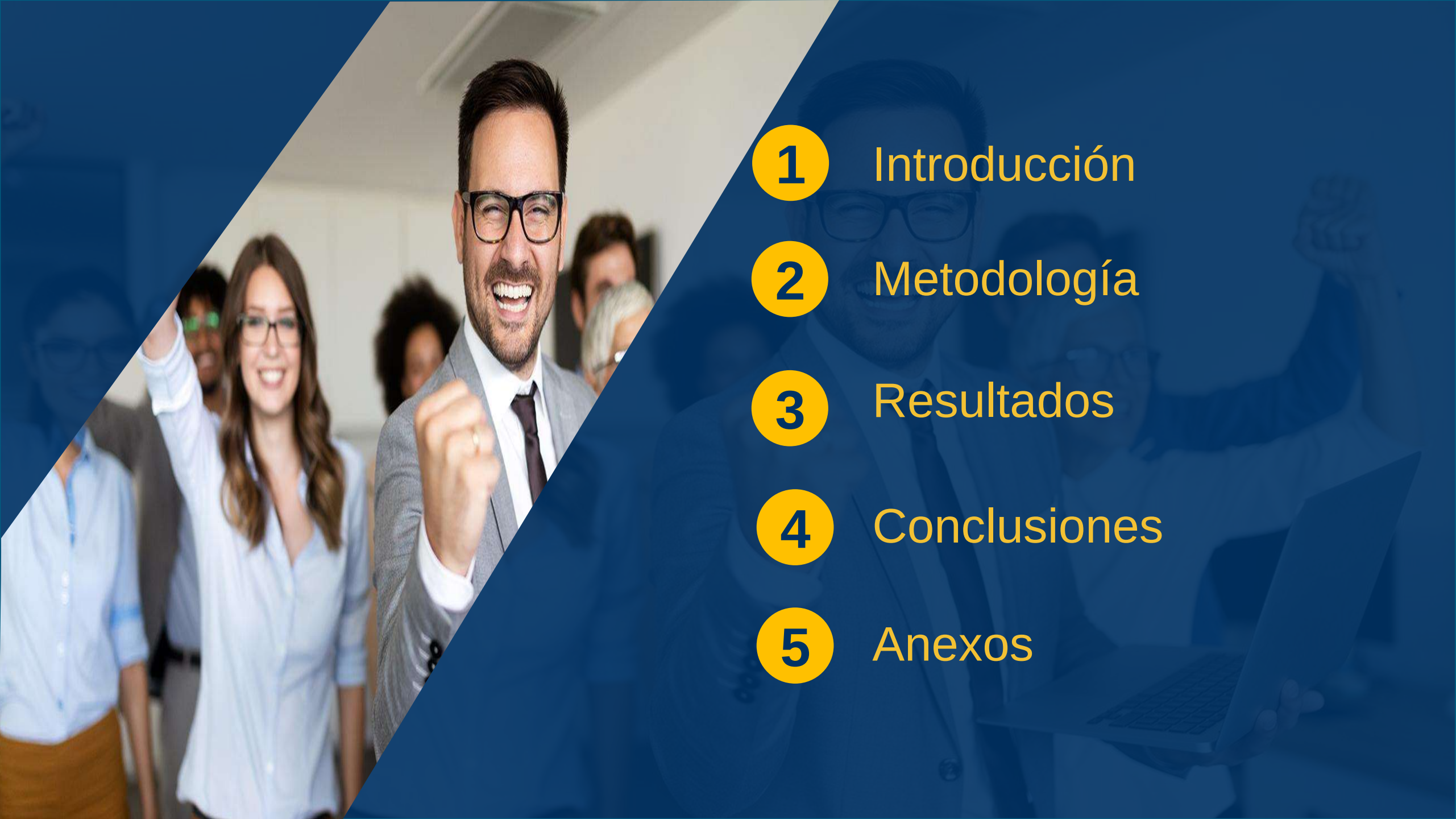




Adaptación Dinámica de Interfaces Empresariales con GRU y DQN bajo Restricciones RBAC

Gutierrez Anco Yesenia Alejandra
Quispe Huarilloclla Jhosef Antony
Turpo Cauna Jose Samuel

Sé Íntegro, Sé Misionero, Sé Innovador



- 1 Introducción
- 2 Metodología
- 3 Resultados
- 4 Conclusiones
- 5 Anexos



El problema: menús estáticos en sistemas de alta densidad

El reto. A medida que los ERP crecen, los menús hiperdensos imponen sobrecarga cognitiva: los usuarios gastan parte de su jornada solo en localizar funciones rutinarias. Aun así, la arquitectura jerárquica permanece estática y genérica, sin ajustarse al rol de cada persona.

Tres limitaciones que la literatura actual no resuelve

1

Markov pierde el contexto

Los modelos probabilísticos simples (cadenas de Markov) no retienen las dependencias a largo plazo de una sesión de trabajo extensa: olvidan de dónde viene el usuario.

2

RBAC ignorado

La reorganización visual ciega promueve funciones a las que el rol no tiene acceso, violando los protocolos de Control de Acceso Basado en Roles.

3

Memoria espacial rota

Los cambios topológicos abruptos destruyen la memoria muscular del usuario, provocando desorientación y rechazo a la herramienta.

La propuesta: predicción GRU + agente DQN con RBAC

Qué hace el sistema. Un ciclo cerrado que observa los logs de navegación, infiere la siguiente función probable con una red GRU y deja que un agente DQN reordene el menú: respetando RBAC y con una transición gradual para no romper la memoria espacial.

G

¿Por qué GRU y no LSTM o Transformer?

Retiene las dependencias largas con muchos menos parámetros y menor huella de memoria: ideal para ejecutarse in-house en el borde (edge) sin latencia perceptible.

Q

¿Por qué DQN y no un RL genérico?

Un RL genérico opera ciego a los permisos. Nuestro DQN integra una máscara RBAC nativa (Q to menos infinito) y optimiza la reducción de clics como recompensa.

Logs de interacción



Predictor GRU



Agente DQN



Interfaz adaptativa



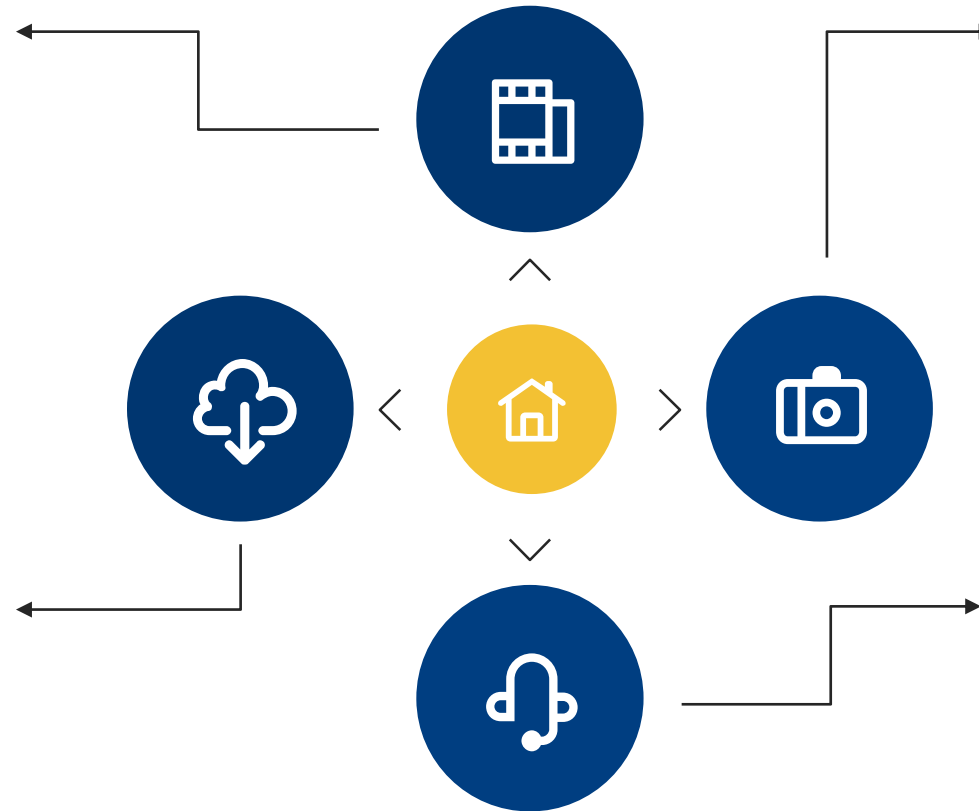
METODOLOGÍA — ARQUITECTURA DEL SISTEMA

Agente DQN + RBAC

Reordena el árbol de menú minimizando clics.
Máscara $Q \rightarrow -\infty$ bloquea ítems no autorizados por rol.

Predictor GRU

Red neuronal recurrente que estima $P(f_{t+1} | f_1 \dots f_t)$. MRR 0.86, supera Cadenas de Markov (0.73).



Logs de Interacción

Recolección pasiva de eventos de navegación del usuario (clics, tiempo, correcciones) durante 4 semanas.

Interfaz Adaptativa

Transición gradual $\beta_{mix}=0.7$: 70% nueva topología + 30% orden original para preservar memoria espacial.



Recolección rigurosa: 4 semanas de logs pasivos

404

sesiones válidas tras el filtrado

6

roles organizacionales simulados

5min

umbral de inactividad para segmentar

Los 6 roles operativos

Administración

Ventas

Compras

Reportes

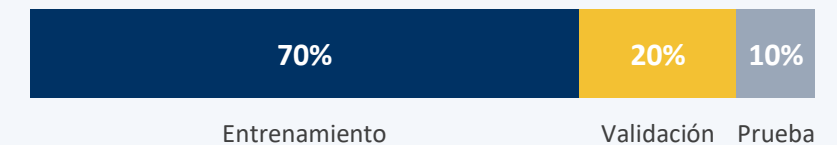
Caja

Logística

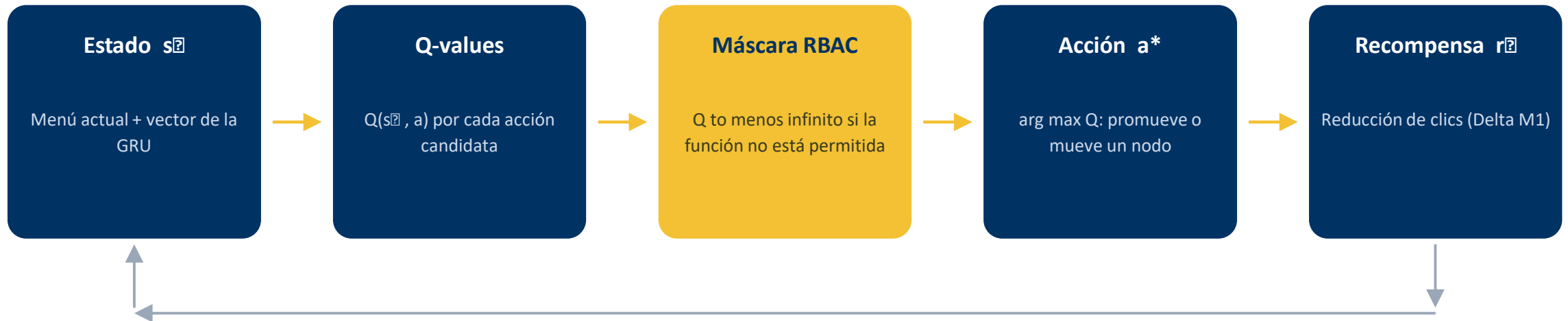
Preprocesamiento

- Recolección 100% pasiva y algorítmica: sin etiquetado manual ni sesgo humano.
- Se filtraron accesos accidentales (permanencia menor a 3 s).
- Se descartaron sesiones incompletas (menos de 2 eventos válidos).

División temporal (sin fuga de datos)



El corazón técnico: un ciclo de decisión con seguridad nativa



Retroalimentación: el menú reestructurado genera el nuevo estado y el ciclo se repite hasta converger.

Reestructuración adaptativa, paso a paso

1

Recorrer acciones

Se evalúa cada movimiento posible sobre el árbol del menú.

2

Aplicar máscara RBAC

Si la acción promueve una función sin permiso del rol, su valor Q se fija en menos infinito.

3

Elegir la óptima

Se selecciona $a^* = \arg \max Q(s,a)$ solo entre las acciones permitidas.

4

Aplicar la acción

Se genera el nuevo árbol de menú reordenado.

5

Mezcla gradual

Se combina el menú nuevo con el original para no perder la referencia visual.

Transición gradual

$$\beta = 0.7$$

Se conserva el 30% del orden original.

Una interfaz matemáticamente óptima no siempre es cognitivamente óptima: frenar la convergencia protege la memoria muscular del usuario.



GRU vs Markov

GRU alcanza MRR 0.86 frente a 0.73 de cadenas de Markov, capturando dependencias a largo plazo.

1

Roles Lineales

Logística: -18.08%. Caja: -16.91%.

2

Roles de Alta Entropía

Administración: -5.69%.
Mayor variedad de tareas = menor optimización posible.

3

Seguridad Preservada

0 violaciones RBAC en 404 sesiones.

4



El DQN gana en todos los roles, pero no por igual

Rol	Estático (M1)	Agente DQN	Mejora
Administración	2.670	2.518	▲ 5.69 %
Ventas	2.000	1.818	▲ 9.07 %
Compras	2.000	1.804	▲ 9.76 %
Reportes	2.000	1.796	▲ 10.19 %
Caja	2.000	1.662	▲ 16.91 %
Logística	2.000	1.638	▲ 18.08 %
Promedio	2.111	1.872	▲ 12.05 %

¿Por qué Administración mejora menos?

Es un rol de **alta entropía**: tareas variadas y poco repetitivas, sin una función que domine la raíz. Y sus **permisos RBAC cruzados** reducen el espacio de reordenamiento permitido sin violar la seguridad.

En el otro extremo: Logística

Sus rutinas son **lineales y muy secuenciales**, así que el agente empuja las funciones más repetidas a la raíz: 18.08% de mejora, casi el doble del promedio.



RESULTADOS — MÉTRICAS CLAVE



Clics (M1)

El agente DQN redujo el promedio de clics de 2.11 a 1.87. Logística alcanzó 18.08% de mejora al priorizar funciones repetitivas.



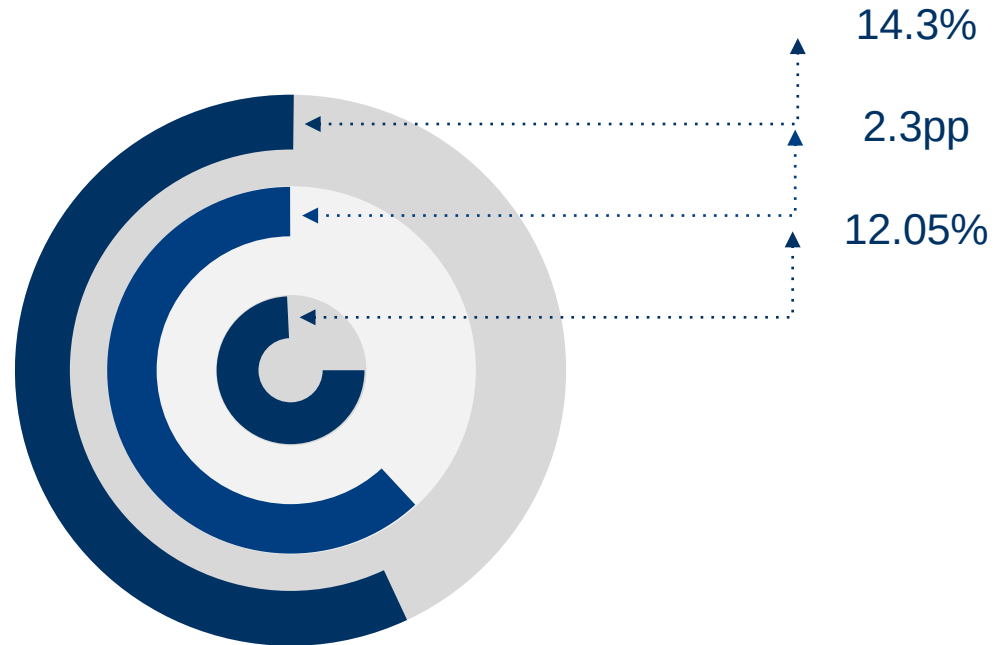
Tiempo (M2)

El tiempo promedio por tarea bajó de 15.40s a 13.20s. La reestructuración reduce la fricción cognitiva, no solo la física.



Errores (M3)

La tasa de desorientación bajó de 4.80% a 2.50%. Menos clics retroactivos gracias a la topología adaptada al rol.



CONCLUSIÓN



Predicción Profunda

GRU supera baselines probabilísticos con MRR 0.86, capturando el contexto real del flujo de trabajo.



Optimización DQN

-12.05% clics, -14.3% tiempo, -2.3pp errores. Política autónoma sin comprometer privacidad.



Seguridad Nativa

Máscara RBAC matemática integrada. 0 violaciones de acceso en todas las sesiones evaluadas.



Trabajo Futuro

Validación con usuarios reales, señales de carga cognitiva implícita y MARL para roles múltiples.



— Parámetros

Learning rate: 1×10^{-3} | Batch size: 32 | Épocas: 150
(early stopping=15)

Optimizador: AdamW ($\beta_1=0.9$, $\beta_2=0.999$)

Regularización: Dropout=0.2, Weight decay= 1×10^{-4}

Framework: PyTorch v2.1.0 | Semilla: 42

Caso de uso — Rol Compras:

Baseline: Compras → Listado → Proveedores → Pagos
Pendientes (4 clics)

DQN adaptado: Compras → Pagos Pendientes → Listado
→ Proveedores (2 clics)

Ahorro: 50% reducción en ruta para la función más
frecuente del rol.